

1. A técnica utilizada se chama Embedding, que consiste no armazenamento de informações relacionadas dentro do mesmo documento.

Nesse caso o histórico de consultas fica embutido diretamente no documento lead. Uma das principais vantagens é o melhor desempenho nas consultas, já que todas as informações ficam serializadas e podem ser acessadas sem a necessidade de relacionamentos complexos.

Uma limitação é o crescimento excessivo do documento ao longo do tempo, se o histórico aumentar muito. Isso pode impactar a performance e dificultar atualizações futuras.

2. Utilizar coleções separadas com referências evita a repetição de informações. Assim, os dados dos clientes e vendedores só ficam armazenados apenas uma vez. Isso facilita a manutenção do sistema, reduz inconsistências e melhora a organização do banco. É torna a expansão mais escalável para futuras expansões.

3. O aggregation Framework permite gerar relatórios e análises diretamente no MongoDB. Para contar leads por origem ou vendedor, pode ser utilizado \$group. Já o \$match serve para filtrar informações específicas, como negociações concluídas.

4. Índices são estruturas que aceleram as consultas no bd. Eles funcionam como o índice de um livro, permitindo localizar informações mais rapidamente. Com os índices o banco evita percorrer todos os documentos da coleção. Isso melhora o desempenho e o tempo de resposta na consulta.

5. A duplicação de dados pode gerar inconsistências e dificultar a manutenção do sistema. Uma modelagem adequada utiliza referências para evitar repetir as informações em vários documentos. Dessa forma qualquer atualização é feita apenas uma vez.

6. Armazenar Tudo dentro de uma única coleção pode simplificar o desenvolvimento. Porém essa abordagem dificulta a organização e a manutenção dos dados. Com o crescimento do sistema as consultas podem se tornar muito complexas e lentas.

7. A paginação é importante porque evita carregar muitos registros no mesmo tempo. Isso melhora o desempenho e a UX. O comando (skip) pula os registros dos páginas anteriores e o limit() define quantos documentos vão ser exibidos por página.

8. A flexibilidade do Mongo DB permite armazenar documentos com estruturas diferentes sem seguir um esquema fixo. Isso facilita alterações e evolução do sistema. Porém, a equipe deve manter um padrão nos dados e sua validação para evitar inconsistências. Assim o banco continua organizado e confiável.

9. O Embedding armazena dados relacionados dentro do mesmo documento, oferecendo consultas mais rápidas. Já o Referencing utiliza referências entre documentos, reduzindo duplicação de informações.

O Embedding é indicado para dados que não mudam muito com frequência. O Referencing é indicado para sistemas maiores e mais evolutivos.

10. Essa integração é eficiente porque ambos (MongoDB e Node.js) trabalham muito bem com dados no formato JSON. Isso facilita o desenvolvimento e a comunicação entre os aplicativos. Além disso, a combinação oferece bom desempenho e escalabilidade. Os usuários recebem respostas mais rápidas.

11. Utiliza o Aggregation Framework para agrupar as negociações por vendedor. O operador \$group pode contar quantas negociações concluídas cada vendedor possui. Depois o \$sort organiza os resultados do maior para o menor.

12. Para isso é importante utilizar índices e uma modelagem adequada. Também é necessário estar duplicando os campos de dados. A escala conta entre Embedding e Referencing ajuda no desempenho das consultas. Além disso, recursos como Sharding podem ser utilizados para distribuir os dados entre vários servidores.

13. Sobre Embedding

14. Comparação entre BDR e BDN

15. O MongoDB é uma boa escolha pois oferece flexibilidade na modelagem dos dados, permitindo adaptações rápidas. Também possui boa escalabilidade para acompanhar o crescimento do sistema.

Além disso, apresenta ótimo desempenho em app web modernas.